TiDB Cloud AI Memory 整理

TiDB Cloud Zero

当前部署

- Aws-us-west-2-f03
- Tidb pool: default pool

管控：

Shadow pool

- 决定 org/project/region，目前只支持
  - Org: TiDB Cloud Zero Prod 1372813089209302377
  - Region: us-west-2-f03

Code block

```
// Shadow Pool 结构定义
type ShadowPool struct {
    ID             uint64
    OrganizationID uint64
    ProjectID      uint64  // ← Cluster 将归属的 Project
    Region         string
    CloudProvider  string
    ServicePlan    string
    // ...
}
```

- ZeroId 实际是 clusterID + uuid 编码
- Claim: 新建集群，复制数据到新集群。https://github.com/tidbcloud/devtier-store-service
  - Tiup dumpling + tiup tidb-lightning

Mem9

两种创建 db 方式

1. tidb cloud zero 方式，返回 api key 即 ZeroId

2. takeoverFromPool 方式，返回 api key 为生成的 uuid

	接口	是否永久	请求	返回信息
tidb cloud zero	https://zero.tidbapi.com/v1alpha1/instances	no	无	Zero id Claim url 连接信息
takeoverFromPool	https://serverless.tidbapi.com/v1beta1/clusters:takeoverFromPool	yes	Tidb cloud api key pool_id （调用者 org 需和 Shadow pool 中 org 一致） root_pswd	Cluster id 连接信息

mem9 内部 tenants 会存连接信息与 api key，api key 直接就查询到连接信息。

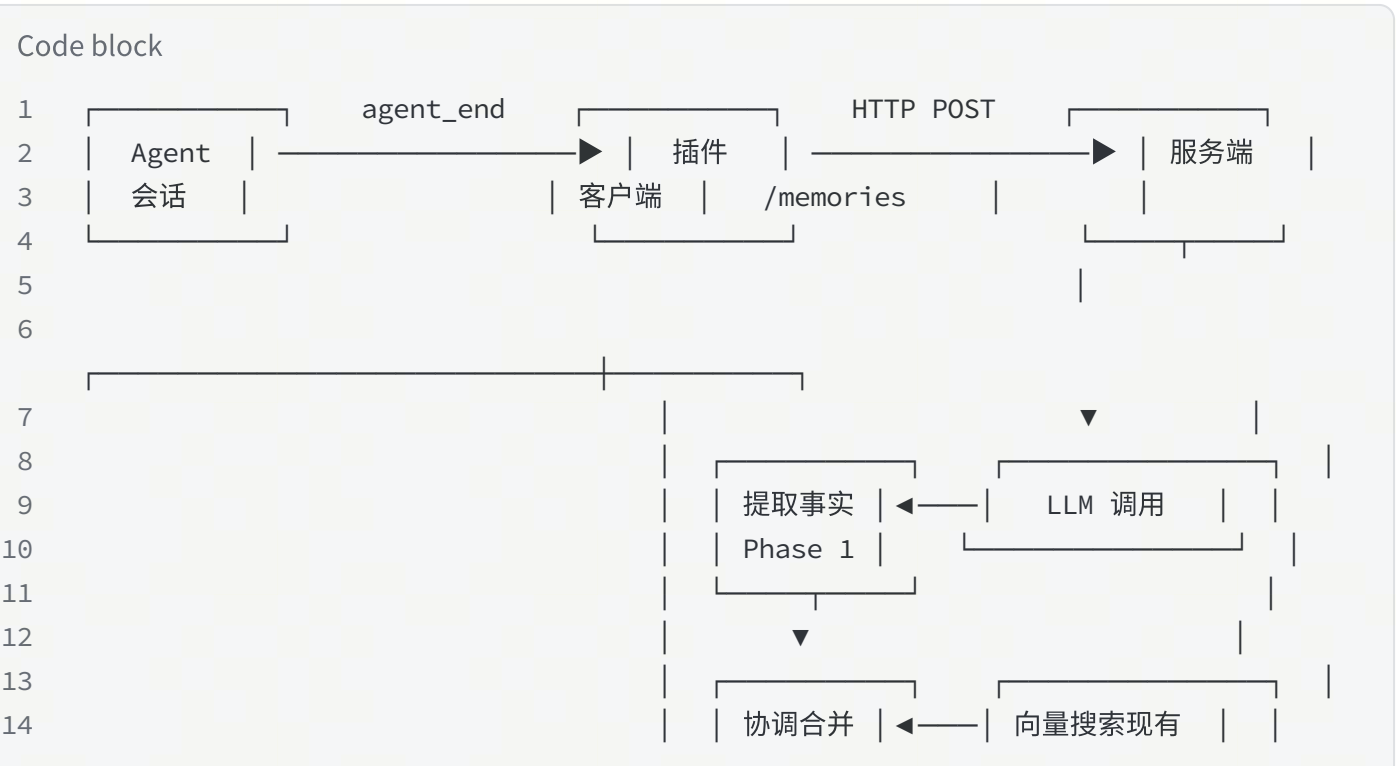
官方部署了一套服务，部署在 https://api.mem9.ai 域名下。

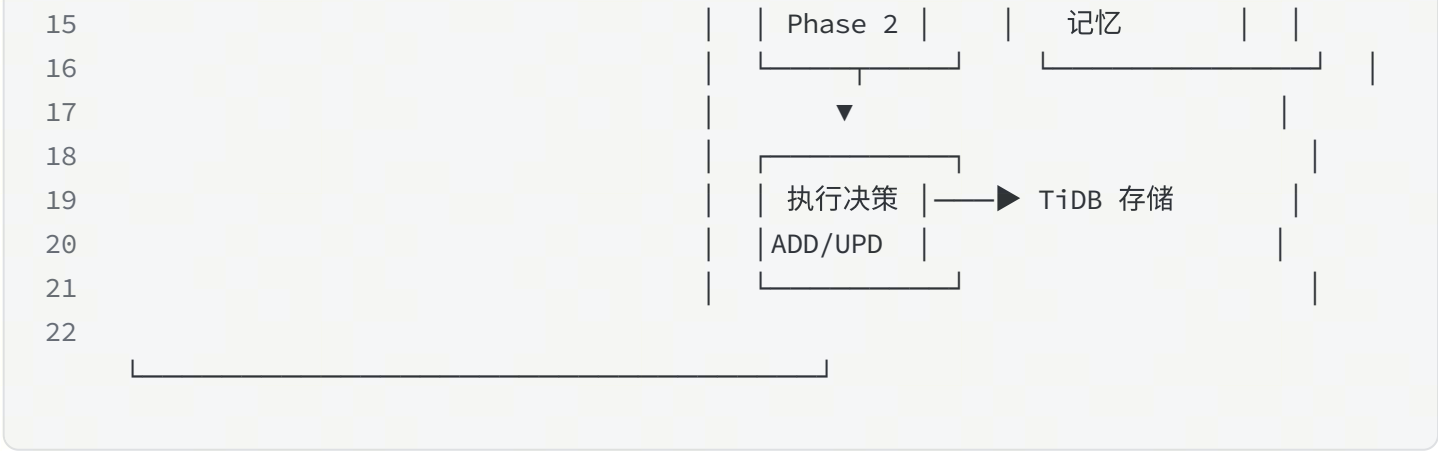
Mem9 原理解析

架构：

客户端+服务端

Code block





客户端存储 mem 时机

- agent_end （hook）： `agent_end` **每轮用户输入触发一次**，不是每会话一次
  a. 格式化，
  b. 上下文剥离：去除之前注入的记忆内容（非对话内容，已存储）
  c. 倒序，200KB，最多20条。
  - 智能模式存入：提取事实+协调合并 (insight 类型)
- before_reset （hook）：
  - 仅保留用户消息，生成摘要。
  - 最后3条用户消息，每条300字符。
  - 直接存入 （pinned 类型）
- 用户调用 tool 要求存入/更新
  - 智能模式存入 （pinned 类型）

服务端智能存入：

1. 上下文剥离：服务端再次剥离，去除之前已存储的 memory
2. 异步保存原始会话
3. 提取事实：基于 LLM 将原始对话转换为**原子化、结构化的事实**。

Prompt example:

Code block

```
systemPrompt := `You are an information extraction engine. Your task is to
identify distinct,
```

```
atomic facts from a conversation AND assign short descriptive tags to each
message.

## Rules — facts

1. Extract facts ONLY from the user's messages. Ignore assistant and system
messages entirely.
2. Each fact must be a single, self-contained statement (one idea per fact).
3. Prefer specific details over vague summaries.
   - Good: "Uses Go 1.22 for backend services"
   - Bad: "Knows some programming languages"
4. Preserve the user's original language. If the user writes in Chinese,
extract facts in Chinese.
5. Omit ephemeral information (greetings, filler, debugging chatter with no
lasting value).
6. Omit information that is only relevant to the current task and has no
future reuse value.
7. If no meaningful facts exist in the conversation, return an empty facts
array.
8. Assign 1-3 short lowercase tags to each extracted fact describing its topic
or
   category. Examples: "tech", "personal", "preference", "work", "location",
"habit".
   Use hyphens for multi-word tags. If no meaningful tags apply, omit the
"tags" field.

## Rules — message_tags

1. Assign 1-3 short lowercase tags to EVERY message (user, assistant, tool,
system).
2. Tags describe the message topic or type. Use your own judgment — there is
no fixed vocabulary.
   Examples: "tech", "work", "personal", "preference", "location", "question",
   "answer", "tool-call", "tool-result", "error", "code", "debug"
3. Tags must be lowercase. Use hyphens for multi-word tags: "tool-call", "tool-
result".
4. Return exactly one array entry per message, in the same order as the input
conversation.
   If a message has no meaningful tags, return an empty array [] for it.

## Examples

Input:
User: Hi, how are you?
Assistant: I'm doing well, thank you! How can I help?
Output: {"facts": [], "message_tags": [[], []]}
```

```
Input:
User: My name is Ming Zhang, I am a backend engineer, mainly using Go and Python.
Assistant: Hi Ming Zhang!
Output: {"facts": [{"text": "Name is Ming Zhang", "tags": ["personal"]}, {"text": "Is a backend engineer", "tags": ["work"]}, {"text": "Mainly uses Go and Python", "tags": ["tech"]}], "message_tags": [["personal", "work", "tech"], ["answer"]]}

Input:
User: I'm debugging a memory leak in our Go service.
Assistant: Let's look at the heap profile. Can you share the pprof output?
User: Here it is: [pprof data...]
Output: {"facts": [{"text": "Debugging a memory leak in a Go service", "tags": ["tech", "debug"]}], "message_tags": [["tech", "debug", "go"], ["tech", "question", "debug"], ["tech", "tool-result", "code"]]}

## Output Format

Return ONLY valid JSON. No markdown fences, no explanation.

{"facts": [{"text": "fact one", "tags": ["tag1", "tag2"]}, {"text": "fact two", "tags": ["tag3"]}], "message_tags": [["tag1", "tag2"], ["tag3"], [], ...]}`
```

4. 协调合并：召回与原记忆比对，判断插入 or 更新 or 删除矛盾信息。由 LLM 决策如何处理

Code block

```
const systemPrompt = `You are a memory reconciliation engine. You manage a knowledge base by comparing newly extracted facts against existing memories and deciding the correct action for each.

## Actions

- **ADD**: The fact is genuinely new information not covered by any existing memory.
- **UPDATE**: The fact refines, corrects, or supersedes an existing memory.
  - Update when: new info is more specific, more recent, or contradicts the old memory.
  - Do NOT update when: old and new convey the same meaning (even if worded differently).
- **DELETE**: The fact directly contradicts an existing memory, making it obsolete.
- **NOOP**: The fact is already captured by an existing memory. No action needed.
```

```
## Rules

1. Reference existing memories by their integer ID ONLY (0, 1, 2...). Never invent IDs.
2. For UPDATE, always include the original text in "old_memory" for audit.
3. For ADD, generate the next sequential integer ID.
4. When in doubt between UPDATE and NOOP, prefer ADD (never lose information — slight duplication is better than missing a new detail).
5. When in doubt between ADD and UPDATE, prefer UPDATE if an existing memory covers a related topic.
6. Preserve the language of the original facts. Do not translate.
7. Each existing memory has an "age" field showing when it was last updated. Use age as a tiebreaker: when a new fact conflicts with an existing memory on the same topic and there is no other signal, older memories are more likely outdated. Age alone is NOT sufficient reason to UPDATE or DELETE — the content must also conflict or supersede the existing memory.

## Tags

Assign 1-3 short lowercase tags to each ADD or UPDATE entry.
Tags describe the topic or category of the memory.
Examples: "tech", "personal", "preference", "work", "location", "habit"
Use hyphens for multi-word tags: "programming-language", "work-tool".
Omit the "tags" field entirely for NOOP and DELETE entries.

## Examples

Example 1 — ADD new information:
  Existing memories: [{"id": 0, "text": "Is a software engineer", "age": "2 months ago"}]
  New facts: ["Name is John"]
  Result: {"memory": [{"id": "0", "text": "Is a software engineer", "event": "NOOP"}, {"id": "new", "text": "Name is John", "event": "ADD", "tags": ["personal"]}]}

Example 2 — UPDATE with more detail:
  Existing memories: [{"id": 0, "text": "Likes to play cricket", "age": "3 weeks ago"}, {"id": 1, "text": "Is a software engineer", "age": "2 months ago"}]
  New facts: ["Loves to play cricket with friends on weekends"]
  Result: {"memory": [{"id": "0", "text": "Loves to play cricket with friends on weekends", "event": "UPDATE", "old_memory": "Likes to play cricket", "tags": ["personal", "habit"]}, {"id": "1", "text": "Is a software engineer", "event": "NOOP"}]}

Example 3 — DELETE contradicted information:
```

```
  Existing memories: [{"id": 0, "text": "Name is John", "age": "5 months ago"}, {"id": 1, "text": "Loves cheese pizza", "age": "3 months ago"}]
  New facts: ["Dislikes cheese pizza"]
  Result: {"memory": [{"id": "0", "text": "Name is John", "event": "NOOP"}, {"id": "1", "text": "Loves cheese pizza", "event": "DELETE"}, {"id": "new", "text": "Dislikes cheese pizza", "event": "ADD", "tags": ["personal", "preference"]}]}

Example 4 — NOOP for equivalent information:
  Existing memories: [{"id": 0, "text": "Name is John", "age": "5 months ago"}, {"id": 1, "text": "Loves cheese pizza", "age": "3 months ago"}]
  New facts: ["Name is John"]
  Result: {"memory": [{"id": "0", "text": "Name is John", "event": "NOOP"}, {"id": "1", "text": "Loves cheese pizza", "event": "NOOP"}]}

Example 5 — Age as tiebreaker for ambiguous conflicts:
  Existing memories: [{"id": 0, "text": "Prefers vim", "age": "1 year ago"}, {"id": 1, "text": "Works at startup X", "age": "8 months ago"}]
  New facts: ["Prefers VS Code", "Works at company Y"]
  Result: {"memory": [{"id": "0", "text": "Prefers VS Code", "event": "UPDATE", "old_memory": "Prefers vim", "tags": ["tech", "preference"]}, {"id": "1", "text": "Works at company Y", "event": "UPDATE", "old_memory": "Works at startup X", "tags": ["work"]}]}

Example 6 — Age does NOT trigger UPDATE without content conflict:
  Existing memories: [{"id": 0, "text": "Name is John", "age": "5 years ago"}]
  New facts: ["Name is John"]
  Result: {"memory": [{"id": "0", "text": "Name is John", "event": "NOOP"}]}

## Output Format

Return ONLY valid JSON. No markdown fences, no explanation.

{"memory": [{"id": "...", "text": "...", "event": "ADD|UPDATE|DELETE|NOOP", "old_memory": "...", "tags": [...]}]}`
```

5. embedding：调用 OpenAI API `text-embedding-ada-002` 模型

数据库

Code block

```
CREATE TABLE memories (
  id              VARCHAR(36) PRIMARY KEY,
  content         TEXT NOT NULL,
```

```
    embedding       VECTOR(1536) NULL,        -- 向量嵌入
    memory_type     VARCHAR(20) DEFAULT 'pinned',  -- pinned/insight/digest
    source          VARCHAR(100),             -- Agent 名称
    tags            JSON,                     -- 标签数组
    metadata        JSON,                     -- 元数据
    agent_id        VARCHAR(100),             -- Agent ID
    session_id      VARCHAR(100),             -- 会话 ID
    state           VARCHAR(20) DEFAULT 'active',  -- active/paused/archived/deleted
    version         INT DEFAULT 1,            -- 乐观锁版本
    created_at      TIMESTAMP,
    updated_at      TIMESTAMP,

    INDEX idx_memory_type (memory_type),
    INDEX idx_state (state),
    INDEX idx_agent (agent_id),
    INDEX idx_session (session_id)
);
```

Mem 召回

客户端召回时机：

- before_prompt_build（hook）：
  - 用户输入触发 LLM 调用前执行, 使用用户当前输入作为搜索查询(跳过少于 5 个字符的短提示)
  - 最多注入 10 条记忆
- 存储记忆时，召回做协调合并：gatherExistingMemories

服务端召回策略：

1. 向量搜索 Top-(limit * 3) → 语义相似度（按最小相似度阈值过滤（默认 0.3））
2. 全文搜索/关键词搜索 Top-(limit * 3) → 精确词匹配
3. RRF（倒数排名融合）合并两者的分数
4. 应用类型权重：pinned=1.5x，insight=1.0x
6. 按融合分数排序，分页返回
7. 添加 relative_age ，相对时间：让 LLM 自主解决时间冲突，如

Code block

```
    (2 days ago) 住在上海
```

```
  （1 year ago）住在北京

 LLM推理：2 天前比 1 年前更新 → 上海是当前居住地
```

客户端召回处理：

- 按类型分组 (Preferences/Knowledge)
- 添加序号、标签、时间提示
- 注入到 LLM 提示上下文

缺点优化点：

1. Rerank：

- 添加轻量级 Cross-Encoder（如 `ms-marco-MiniLM-L-6-v2`）
- 或使用 LLM-based Rerank（成本较高）

2. 无缓存层

Code block

```
// 1. 查询缓存：相同查询直接返回queryCache: map[hash]cachedResult// 2. 向量缓存：避免重复嵌入embeddingCache: map[text]vector// 3. 结果缓存：热门记忆常驻内存hotMemories: LRU Cache
```

3. 缺少学习机制

4. 记忆分层

Code block

```
人类记忆模型（2026 年 AI 应用）：
┌─────────────────────────────────────────┐
│  Episodic Memory（情景记忆）            │
│  - 具体事件："2026-01-15 设计了用户系统" │
│  - 时间、地点、上下文完整               │
├─────────────────────────────────────────┤
│  Semantic Memory（语义记忆）            │
│  - 抽象知识："用户系统需要认证模块"      │
│  - 去时间化，可复用                     │
└─────────────────────────────────────────┘
```

短期（1-3 个月）：工程优化

改进项	技术	预期收益
嵌入模型升级	text-embedding-3-large	+5% 召回率
添加 Reranker	bge-reranker-v2-m3	+10% 精度
多级缓存	Redis + 本地缓存	-50% 延迟
动态阈值	查询自适应	+5% F1

中期（3-6 个月）：架构升级

改进项	技术	预期收益
Agentic RAG	规划 + 多步检索	复杂查询 +20%
记忆分层	Episodic/Semantic	长期记忆 +15%
流式更新	Kafka + 实时索引	新鲜度实时
评估体系	RAGAS 2.0	可量化优化

长期（6-12 个月）：前沿探索

改进项	技术	预期收益
Matryoshka 嵌入	多维度统一表示	存储 -50%，速度 +30%
生成式检索	DSI, LLM-based	端到端优化
多模态记忆	代码 + 文本 + 图像	覆盖更多场景
个性化学习	在线学习用户偏好	用户体验 +20%

5. 记忆提取优化

短期改进

```typescript
// 1. 增加重要消息检测
function detectImportantMessages(messages: IngestMessage[]): number[] {
  // 检测包含关键信息的索引
  // 如：包含"决定"、"确定"、"结论"等关键词的消息
}

// 2. 混合选择：重要消息 + 最新消息
const important = detectImportantMessages(allMessages);
const recent = selectMessages(allMessages, maxBytes / 2);
const selected = mergeUnique(important, recent);
```

中期改进

```typescript
// 3. 增量提取
// 不是只在 agent_end 提取，而是每 N 条消息触发一次
if (messages.length % 10 === 0) {
  await incrementalExtract(messages);
}
```

Claw mem 解析

架构

基于 subagent 执行提取记忆，reconcile 记忆等操作.

存储：一个 session -> 1(type:conversation，完整对话)+N（type:memory，具体记忆） github issue

召回：全文检索 memory 类型 github issue

二、记忆的 Kind 分类（5 种）

根据 schema.md:33-42，`kind:*` 标签定义记忆的**语义类别**：

Kind	标签	用途说明
Core fact	`kind:core-fact`	稳定的事实，随现实变化而更新
Convention	`kind:convention`	约定的规则或策略
Lesson learned	`kind:lesson`	纠正、复盘或值得保留的错误
Skill blueprint	`kind:skill`	可重复的工作流或操作手册
Active task	`kind:task`	需要持续跟踪的进行中任务

使用场景对照（schema.md:44-51）：

触发场景	使用的 Kind
用户纠正错误假设	`kind:lesson`
双方约定规则	`kind:convention`
关于用户/项目的稳定事实	`kind:core-fact`
整理出可重复的工作流程	`kind:skill`
需要跟踪的持续性工作	`kind:task`

三、Topic 维度（开放式标签）

`topic:*` 标签用于**主题分类**，是开放式的、可自定义的：

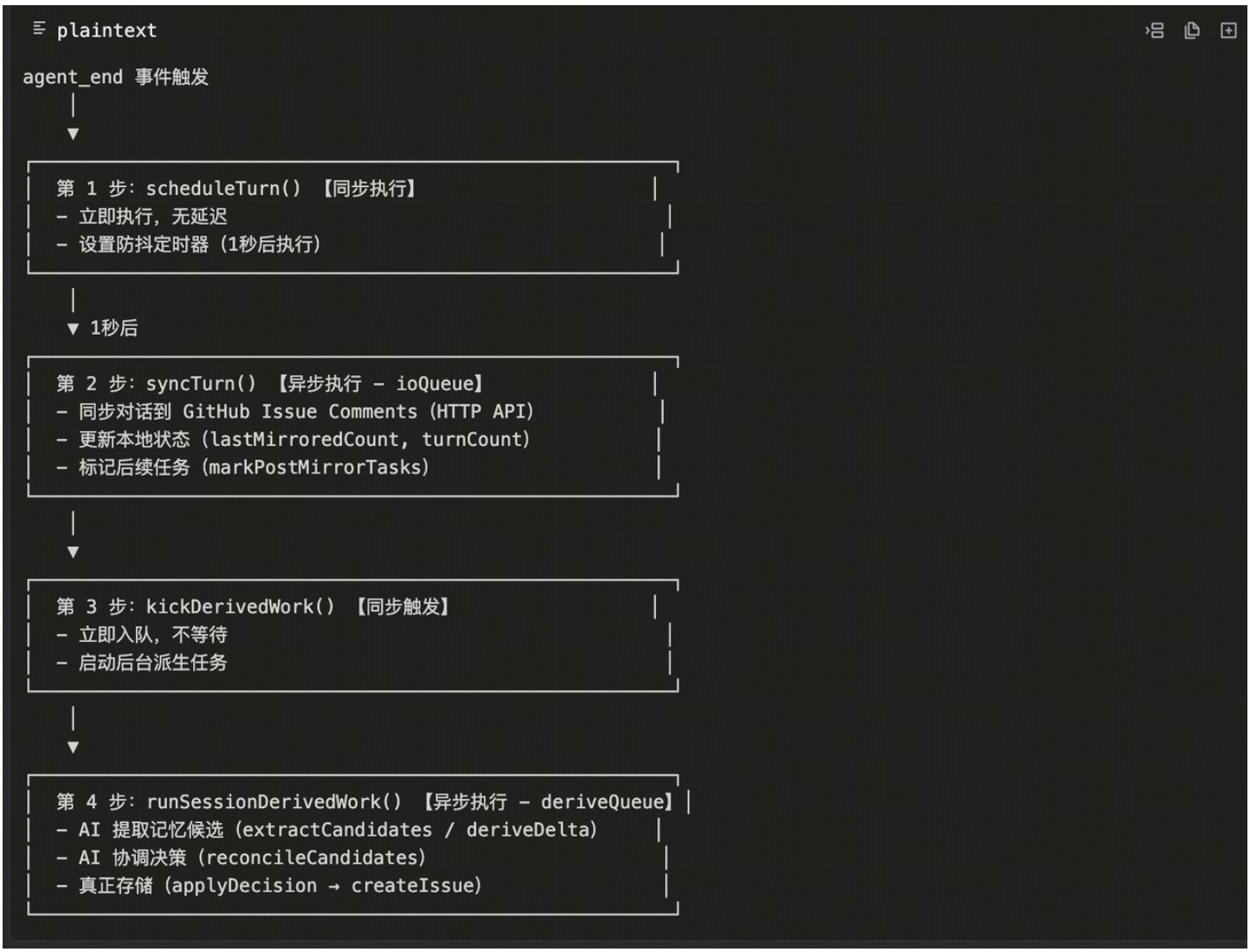
- 一个记忆可以有 **0-3 个 topic**
- 用于横向关联不同 kind 的记忆
- 例如：`topic:preferences`、`topic:deployment`、`topic:refactoring`

记忆提取（0.1.16）

- session_end, reset

- Final summary: 基于 digest 最终生成 summay

- agent_end
  - scheduleTurn: 1秒内无 agent_end 就触发，有就合并处理
  - 异步 syncTurn: 从文件拉 message，基于游标过滤重复 message；最后触发 kickDerivedWork 启动后台任务
    - 同一个 session 串行
    - 不同 session 并行
  - 异步 runSessionDerivedWork （仍然同一个 session 串行，不同 session 并行）



runSessionDerivedWork 详解

1. 提取 diget （subagent AI）： 滚动累计 digest

Code block

```
1    "Maintain a rolling digest of the conversation below.",
2         'Return valid JSON only in the form {"digest":"...","title":"..."}',
```

```
      "Update the digest so it accurately represents the conversation so far in 4-8 concise factual sentences.",
      "Focus on decisions, constraints, preferences, open workstreams, and concrete outcomes worth carrying forward.",
      "Use the anchor messages only for context resolution. The new messages are the only part that must be incorporated now.",
      "Title is optional. If provided, keep it under 50 characters and accurately describe the overall conversation.",
      "",
      "<previous-digest>",
      previousDigest?.trim() || "None.",
      "</previous-digest>",
      "",
      "<anchor-messages>",
      anchorMessages.length > 0 ? fmtTranscriptFrom(anchorMessages, anchorStart) : "None.",
      "</anchor-messages>",
      "",
      "<new-messages>",
      fmtTranscriptFrom(deltaMessages, deltaStart),
      "</new-messages>",
    ].join("\n");
```

2. 提取记忆候选（subagent AI）： ai 生成 label/kind 等内容，后续召回可用于打分

Code block

```
 const message = [
      "Extract atomic durable memory candidates from the conversation delta below.",
      'Return JSON only in the form {"candidates": [{"title":"...","detail":"...","kind":"...","topics": ["..."],"evidence":"..."}]}.',
      "Only extract durable facts, preferences, decisions, constraints, workflows, and ongoing context worth remembering later.",
      "Use the anchor messages and rolling digest only for context resolution. The new messages are the only source that may add new candidates now.",
      "Each candidate must represent one durable fact. Split independent facts into separate candidates.",
      "Do not extract temporary requests, tool chatter, startup boilerplate, or summaries about internal helper sessions.",
      "Kind and topics are optional. Keep them short, reusable, and low-cardinality.",
      "Evidence is optional. If present, keep it short and quote-free.",
      "Prefer an empty candidates array when nothing durable was added.",
      "",
```

```
        "<rolling-digest>",
        digestText?.trim() || "None.",
        "</rolling-digest>",
        "",
        "<anchor-messages>",
        anchorMessages.length > 0 ? fmtTranscriptFrom(anchorMessages, anchorStart) : "None.",
        "</anchor-messages>",
        "",
        "<new-messages>",
        fmtTranscriptFrom(deltaMessages, deltaStart),
        "</new-messages>",
      ].join("\n");
```

3. Reconcile 记忆候选（subagent AI）：将新提取的记忆候选与现有记忆对比，进行决策
   a. 获取候选记忆
   b. 召回相关记忆
   c. AI 决策（prompt 如下）
   d. 执行决策：保存新记忆/不保存新记忆/删除旧记忆

Code block

```
    "Reconcile extracted durable memory candidates against existing memories.",
        'Return JSON only in the form {"save":[{"title":"...","detail":"...","kind":"...","topics":["..."]}],"stale":["memory-id"]}.',
        "Use save only for candidates that should become durable memories after comparing them with existing memories.",
        "If a candidate is already fully covered by an existing memory, omit it from save.",
        "Use stale only when a candidate clearly supersedes or invalidates an existing memory.",
        "Do not stale memories just because they overlap or are related. Prefer keeping both when they can coexist.",
        "Keep each save item atomic and durable.",
        "",
        "<candidates>",
        candidateBlock,
        "</candidates>",
        "",
        "<matching-existing-memories>",
        existingBlock,
        "</matching-existing-memories>",
      ].join("\n");
```

4. 存储 git backend，git backend 会 embedding

记忆提取（0.1.17）

只在会话结束时提取，handleFinalize hook

Code block

```
`Write the final issue summary and extract durable memory candidates from the conversation below.
Return valid JSON only in the form {"summary":"...","title":"...","candidates":[{"title":"...","detail":"...","kind":"...","topics":["..."],"evidence":"..."}]}.
The summary should be concise, factual, and written in 2-4 sentences.
Do not include markdown, bullet points, or analysis.
Title rules:
- Under 50 characters, accurately describe the main topic or task.
- Should let someone immediately know what the conversation is about.
- Must be in the same language as the majority of the conversation content.
- Good: precise, descriptive, specific. Bad: vague, overly creative, generic.
Candidate rules:
- Extract only durable facts, preferences, decisions, constraints, workflows, and ongoing context worth remembering later.
- Each candidate must represent one durable fact. Split independent facts into separate candidates.
- Prefer a concise explicit title for each candidate whenever the fact can be named clearly.
- Candidate titles and details must be in the same language as the majority of the conversation content.
- Do not extract temporary requests, tool chatter, startup boilerplate, or summaries about internal helper sessions.
- Reuse existing schema labels when one already fits.
- If no existing kind or topic fits, create one short stable machine-readable label instead of a translated or near-duplicate variant.
- Keep kind and topic labels short, reusable, low-cardinality, and machine-readable.
- Evidence is optional. If present, keep it short and quote-free.
- Prefer an empty candidates array when nothing durable was learned.
Current schema to reuse first:
<current-schema>
Kinds:
- kind:core-fact
- kind:convention
Topics:
- topic:redis
- topic:rate-limits
```

```
</current-schema>
Prefer these existing labels whenever they fit. Only create a new label when none of the current labels matches the fact you are storing.
<conversation>
xxx
</conversation>
```

召回

触发

- before_prompt_build: 召回注入系统 prompt
- before_agent_start：召回注入 prompt （根据历史消息）

流程：

1. 粗排：search github，查询相关 issue（git backend 会基于向量检索+全文见色）

Code block

```
  private async searchViaBackend(query: string, limit: number): Promise<ParsedMemoryIssue[]> {
    const repo = this.client.repo();
    if (!repo) throw new Error("ClawMem memory recall requires a configured repo.");
    const qualified = buildMemorySearchQuery(query, repo);
    const batch = await this.client.searchIssues(qualified, { perPage: Math.min(100, Math.max(limit * 3, 20)) });
    return batch
      .map((issue) => this.parseIssue(issue))
      .filter((memory): memory is ParsedMemoryIssue => memory !== null && memory.status === "active")
      .slice(0, limit);
  }
```

2. 精排：对 title/detail/kind/topics 进行多粒度匹配与重叠率评分，用于回退或排序。（还未调用）

Code block

```
export function scoreMemoryMatch(memory: ParsedMemoryIssue, rawQuery: string): number {  const query = normalizeSearch(rawQuery);  if (!query) return 0;  const idx = buildSearchIndex(memory);  const tokens = searchTokens(query);  const queryTokenSet = new Set(tokens);    let score = 0;    // ========== 1. 完整匹配加分（高权重）==========  if (idx.title.includes(query)) score += 18;    // 标题完整包含  if (idx.detail.includes(query)) score += 12;          // 详
```

```
情完整包含  if (idx.kind?.includes(query)) score += 8;            // kind 完整包含  for (const topic of idx.topics) {    if (topic.includes(query)) score += 10;             // topic 完整包含  }    // ========== 2. Token 级别匹配（中权重）==========  for (const token of tokens) {    if (idx.title.includes(token)) score += 4;    if (idx.detail.includes(token)) score += 2;    if (idx.kind?.includes(token)) score += 3;    if (idx.topics.some((topic) => topic.includes(token))) score += 3;  }    // ========== 3. Token 集合重叠率（Jaccard）==========  score += overlapRatio(queryTokenSet, titleTokenSet) * 10;  score += overlapRatio(queryTokenSet, detailTokenSet) * 6;  score += overlapRatio(queryTokenSet, kindTokenSet) * 6;  score += overlapRatio(queryTokenSet, topicTokenSet) * 8;    // ========== 4. 字符 Bigram 重叠率（模糊匹配）==========  const queryBigrams = charBigrams(query);  score += overlapRatio(queryBigrams, charBigrams(idx.title)) * 6;  score += overlapRatio(queryBigrams, charBigrams(idx.detail)) * 3;    return score;}
```

Claude Code Mem

Mem0

分类：

- semantic： 事实
- episodic： 发生过的事
- procedural：应该怎么做

SuperMemory

Backend 闭源，只有 plugin 暴露

Memory 设计

> 不要只存事实，还要存经验和流程

> - 不要只会新增，要支持覆盖、失效、归档
> - 不要让所有记忆都同权，必须有优先级和时间衰减
> - 不要让记忆不可见，最好支持审阅和删除

分类

- fact：事实
- episodic：发生过的事
- procedura：流程/经验/约定

kind：横向关联记忆

- Skill
- Task
- Lesson
- Workflow
- Preference
- Profile
- note

Kind 为主：因为用户会问找我偏好/找之前xxx的坑之类。kind 为主好召回

Scope:

- User：参考mem9/clawmem 都是面向用户 （prompt conext）
- agent：TODO，让 agent 自己进化。如内部策略，系统行为约束，workflow。通过实施结果，用户反馈不停进化？ (system prompt)

1. 记忆抽取
    a. 后台进行
    b. Tool call：对话事实进行
2. 召回：
    a. 时机：每轮对话？pre

b. semantic search + 结构化过滤（标签/关键词/）全文检索

分层记忆：

- Working memory：当前这轮对话的，比如临时上下文，任务中间状态
- Session memory：session 中的 memory
- Long-term memory: 数据库
- Org memory (知识库)：数据库

关键决策点：

- 架构：客户端+服务端，服务端提供插入/查询/更新/删除操作，封装 embedding
- 存储
  - 时机
  - 提取方式：LLM 事实提取+reconcile
  - 分类
- 召回
  - 粗排：向量检索+全文检索+RRF
  - 精排1：算法
  - 精排2：加权

设计 temperato